Casino Card Game Suite

CS-A1123 Basics in Programming Y2
Aalto University

Ang Yong Yi, Jatlyson
Student ID: 104083758

General Information

1. General Description and Difficulty Level

This project implements a multi-game casino card game suite in Python using the PyQt6 graphical framework. The primary game is the Finnish Deck Casino (Kasino-Kortipelli) card game, with Blackjack and Baccarat included as additional games. Players interact through a unified graphical lobby from which they can select a game and manage their virtual session points or bankroll.

The target difficulty level is Hard. The primary Deck Casino game implements all Hard-level requirements: full graphical UI, save/load of game state at any point, and computer opponents with genuine strategy. Blackjack and Baccarat are implemented as supporting games to complete the casino suite.

What makes this project unique

- Primary Deck Casino (Kasino-Kortipelli) game: Full implementation of the Finnish card-taking game with multi-player support, sweep detection, special card values, and a strategic AI opponent.
- Strategic AI for Deck Casino: The computer opponent avoids leaving easy sweeps, prioritises high-value cards (Diamond-10, Spade-2, Aces), and plans card placements to set up future takes.
- Multi-game lobby architecture: Three games share a common engine layer (Deck, Hand, Player/bankroll), reducing code duplication while allowing each game to define its own rules as a subclass of the abstract Game class.
- Complete game-state saving: The entire game situation (all hands, table cards, scores, whose turn it is) can be saved and restored at any point mid-round.

2. Use Case Description and Draft of the User Interface

User Interface Overview

The application is a single-window PyQt6 application. A persistent left-side sidebar shows the player's name, current points/bankroll, and navigation buttons (Lobby, Settings). The main area swaps between a Lobby View and individual Game Views.

Screen/ Widget	Description
Lobby View	Grid of game cards (Deck Casino, Blackjack, Baccarat) with a Play button each. Shows current balance/points prominently.
Deck Casino View	Table cards shown centrally. Each player's hand displayed. Current player highlighted. Action buttons: Take Cards, Place Card. Sweep counter and score panel shown.
Blackjack View	Dealer hand top, player hand bottom, action buttons: Hit, Stand, Double Down, Split. Bet input and chip selector above player hand.
Baccarat View	Punto and Banco hands side by side. Bet panel with Punto / Banco / Tie buttons. Results announcement overlay after each round.
Game History View	Shows per-round results for the current session: game played, result, and point/bankroll change. Simple scrollable list.

Use Case: Player Plays a Round of Deck Casino

1. **Launch:** Player starts the program. GameManager initialises and loads any saved game state. Lobby View is displayed.
2. **Select game:** Player clicks 'Play Deck Casino'. GameManager.load_game('casino') instantiates DeckCasinoGame. The Deck Casino View is pushed to the main area. Players enter their names and choose the number of opponents (human or AI).
3. **Deal:** DeckCasinoGame.start_round() shuffles the deck and deals 4 cards to each player and 4 cards face-up to the table. The remaining cards form the stock pile.
4. **Player turn:** The active player sees their hand and the table cards. They select a card from their hand. The game logic automatically computes all valid takes using the CasinoTakeAlgorithm. The valid combinations are highlighted; the player selects which take to use, or places the card on the table if no take is desired or available.
5. **Card replenishment:** After each action, the player draws a new card from the stock (while stock is non-empty).
6. **Sweep detection**: If a take clears all table cards, DeckCasinoGame.check_sweep() flags a sweep for that player.
7. **AI turn:** When it is a computer opponent's turn, AIOpponent.decide_action() evaluates the table and hand using the strategy described in Section 6, then takes or places a card accordingly.
8. **Round end:** When all players run out of cards, the last player to take receives any remaining table cards. DeckCasinoGame.calculate_round_points() scores all collected piles (sweeps, aces, card count, spades, Diamond-10, Spade-2). Points are added to cumulative totals.
9. Game end: Play continues until a player reaches 16 points. The winner is announced. Players can start a new game or return to the lobby.
10. **Save & Return:** At any point the player can save the entire game state (all hands, table, stock, scores, turn order) to a JSON file and reload it later, session record is appended to history. Player can return to the lobby or play again.

Use Case: Player Plays a Round of Blackjack

1. **Launch:** Player starts the program. GameManager initialises and loads the save file (bankroll). Lobby View is displayed.
2. **Select game:** Player clicks 'Play Blackjack'. GameManager.load_game('blackjack') instantiates BlackjackGame. The Blackjack View is pushed to the main area.
3. **Place bet:** Player selects chip value (e.g. 25 EUR) and clicks the table to stack chips. BettingSystem.validate_bet() checks against min/max. Bet is confirmed.
4. **Deal:** Player presses 'Deal'. BlackjackGame.start_round() deals two cards to player and dealer (one face-down). Hand objects are populated; the GUI updates card widgets.
5. **Player actions:** Player clicks 'Hit'. BlackjackGame.hit() calls Deck.deal(), adds Card to Hand, Hand.get_value() is recomputed. If bust, round ends immediately.
6. **Dealer plays:** Player stands. AIDealer.decide_action() is called in a loop; dealer hits until value >= 17 (basic strategy). Each card is animated with a short QTimer delay.
7. **Resolve:** BlackjackGame.end_round() compares hand values. BettingSystem.calculate_payout() returns the correct multiplier. Player.win() or Player.lose() updates bankroll. Result is displayed.
8. **Save & return:** Session record is appended to history. Player can return to the lobby or play again.

Use Case: Player Plays a Round of Baccarat

1. **Select game:** Player clicks 'Play Baccarat'. GameManager.load_game('baccarat') instantiates BaccaratGame. The Baccarat View is pushed to the main area.
2. **Place bet:** Player selects a bet on Punto (Player), Banco (Banker), or Tie using the bet panel. BettingSystem.place_bet() validates the amount against the bankroll.
3. **Deal:** Player clicks 'Deal'. BaccaratGame.start_round() calls Deck.deal() four times — two cards to Punto and two to Banco — alternating as per standard dealing order. Both hands are displayed face-up.
4. **Third-card rule:** BaccaratGame.draw_third_card() applies the deterministic rule table: if either hand is a natural (8 or 9), no further cards are drawn; otherwise Punto draws if its value is 0–5, and Banco draws according to the fixed Banco rule table. Any third cards are animated with a QTimer delay.
5. **Resolve:** BaccaratGame.end_round() compares the final hand values (mod 10). The hand closer to 9 wins. BettingSystem.calculate_payout() applies the correct multiplier (Punto win: 1:1; Banco win: 1:1 minus 5 % commission; Tie: 8:1). Player.win() or Player.lose() updates the bankroll and the result overlay is shown.
6. **Save & Return:** Session record is appended to history. Player can return to the lobby or play again.

3. Plan for the Structure of the Program

The program is divided into three layers: Model (game logic and data), Controller (GameManager, AI), and View (PyQt6 widgets). This separation ensures game logic can be tested without a GUI and views can be swapped without changing rules.

Core Model Classes

Class	Description
Card	Immutable value object: suit, rank, numeric value. Knows its hand value and table value (with special rules for Aces, Diamond-10, Spade-2).
Deck	Holds an ordered list of Card objects. Provides shuffle and deal operations.
Hand	Aggregates Cards for one participant. Computes hand value; detects bust (Blackjack), and poker hand ranks.
Player	Holds the player's name, score/bankroll, and current bet/collected cards. Provides place_bet, win, lose, and add_to_collection operations.
AIOpponent	Strategy-based engine for Deck Casino. Decides which card to play and which table cards to take using priority rules described in Section 6.
AIDealer	Rule-based engine for Blackjack (basic strategy lookup table) and Baccarat (deterministic third-card rules).
BettingSystem	Validates bets and calculates payouts per game rules (Blackjack, Baccarat). Not used for Deck Casino (point-based).

CasinoTakeAlgorithm	Core algorithm class. Given a played card and a list of table cards, enumerates all valid single-card takes and multi-card sum combinations. Returns all valid take sets.
Game (abstract)	Base class for all games. Holds references to Player(s), Deck, and Hand(s). Defines abstract methods: start_round, end_round, get_valid_actions.
DeckCasinoGame	Extends Game. Implements the full Deck Casino rules: dealing, card taking via CasinoTakeAlgorithm, sweep detection, special card scoring, and round/game point calculation.
BlackjackGame	Extends Game. Implements hit, stand, double down, and split. Manages a split hands list.
BaccaratGame	Extends Game. Manages Punto/Banco hands, third-card drawing rule, and tie detection.
GameManager	Central controller. Holds a dict of available games, tracks the active game, manages turn order, and delegates save/load to FileManager.

UML Class Diagram

UML class diagrams for both the Deck Casino game and the full suite are provided as separate attachments to this plan.

UI Classes

All UI classes are PyQt6 QWidget subclasses: MainWindow (application frame, sidebar, QStackedWidget), LobbyView, DeckCasinoView, BlackjackView, BaccaratView, and BettingPanel. Each game view connects Qt signals (button clicks) to the corresponding Game method via GameManager, then refreshes itself from the updated model state.

4. Data Structures

- **list** — Used in Deck (ordered Cards; pop() for dealing), Hand (accumulates dealt cards), and Table (cards currently on the table in Deck Casino).
- **dict** — Used in GameManager to map game names to Game instances, AIDealer basic-strategy lookup table {(player_total, dealer_upcard): action}, and JSON save/load structures.
- **set** — Used in CasinoTakeAlgorithm when enumerating valid card combinations to avoid duplicate subsets.
- **tuple** — Card rank/suit pairs when iterating; immutable, appropriate for fixed data.
- **JSON (via json module)** — Persistent storage of full game state (hands, table, scores, turn index) and bankroll. Human-readable; no extra dependencies.

Plain Python lists are sufficient because the largest collection is a 6-deck shoe (312 cards). random.shuffle will be applied to the list in-place.

5. Files and File Formats

The program uses a single persistent JSON text file (save_data.json) in the application directory. JSON was chosen because the data is nested (player info, all hands, table cards, collected piles, sweep counts, round scores) and Python's built-in json module handles it without extra dependencies. For Deck Casino, the full game state, including each player's

hand, the table cards, the stock pile, the current turn index, and cumulative scores is serialised so the game can be resumed exactly from where it was left.

FileManager reads this file on startup (if it exists) and writes it whenever the player explicitly saves. If the file does not exist, defaults are used (bankroll = 1000 EUR for betting games; fresh Deck Casino game). No binary files are needed; card graphics are drawn programmatically with QPainter.

6. Algorithms

Casino Take Algorithm (CasinoTakeAlgorithm)

This is the core algorithm for the Deck Casino game. Given a card played from hand (with its hand value) and a list of table cards (with their table values), the algorithm must find all valid sets of table cards that the player may take.

A valid take is a non-empty subset of table cards whose values either: (a) all equal the played card's hand value (single-value matches), or (b) sum to the played card's hand value (sum matches). Both types can be combined: any subset of table cards whose total equals the played card's hand value is a valid take.

Algorithm steps: (1) Assign table values to all table cards (special rules for Ace=1, Diamond-10=10, Spade-2=2 on the table). (2) Use the played card's hand value as the target T. (3) Enumerate all non-empty subsets of the table cards using itertools.combinations across all lengths 1..n. (4) For each subset, check whether the sum of table values equals T. (5) Return all valid subsets. The player selects one of these subsets (or chooses to place the card instead).

Complexity: O(2^n) where n is the number of table cards. In practice n is at most ~4-8 cards during play, making this tractable. The algorithm is implemented in CasinoTakeAlgorithm.get_valid_takes(played_card, table_cards) and returns a list of frozensets.

AI Opponent Strategy (Deck Casino)

The AI opponent (AIOpponent) uses the following priority rules on its turn:

1. Sweep opportunity: If any card in hand can take all table cards at once, always do so (guaranteed sweep).
2. High-value card protection: Prefer using Diamond-10 (hand value 16) or Spade-2 (hand value 15) to take cards rather than placing them, to prevent other players from using them.
3. Maximise cards taken: Among valid takes, prefer the take that collects the most cards.
4. Special card collection: Prioritise taking Aces (1 point each), Diamond-10, and Spade-2 from the table when possible.
5. Avoid easy sweeps for opponents: When placing a card, avoid leaving a single-card table or a table whose total matches a card likely in opponents' hands.
6. Card placement: When no good take is available, place the card with the lowest hand value (to minimise gift potential).

Hand Value Calculation (Blackjack)

Cards 2-10 contribute their face value; Jack, Queen, King = 10; Ace = 11 unless total > 21, then 1. Implementation: sum non-ace values, count aces, add 11 for first ace, reduce each 11 to 1 while total > 21. O(n) in hand size.

Baccarat Third-Card Rule

Baccarat uses a deterministic rule table: if either hand has a natural (8 or 9), no further cards are drawn; otherwise Punto draws if its value is 0-5, and Banco draws according to a fixed rule table indexed by Banco's value and Punto's third card. Implemented as conditional checks in BaccaratGame.draw_third_card().

7. Testing Plan

System-Level Testing

The final program will be tested by playing full rounds in each game and verifying: (a) Deck Casino rules applied correctly; (b) all valid takes enumerated and no invalid takes allowed; (c) sweep detection works correctly; (d) point calculation correct at round and game end; (e) game state saved and reloaded exactly mid-round; (f) bankroll changes correctly in Blackjack/Baccarat; (g) AI does not leave easy sweeps.

Feature	Test Plan
CasinoTakeAlgorithm	Use the worked example from the rules: table has 6 specific cards, hand has 4 cards. Assert exact valid take sets match expected output.
Sweep detection	After a take that clears the table, assert sweep count increments. Partial take does not increment.
Point calculation	Set up known collected piles: assert correct points for most cards, most spades, aces, Diamond-10, Spade-2, and sweeps.
Special card values	Assert Ace hand value = 14, table value = 1. Diamond-10 hand = 16, table = 10. Spade-2 hand = 15, table = 2.
Game state save/load	Start a round, make 2 moves, save, reload. Assert all hands, table, stock, scores, and turn index match exactly.
Blackjack hand value	Ace+10 = 21 (natural); ace+5+7 = 13 (ace demoted); 5+5+5+7 = bust (22).
Betting validation	Bet of 0 EUR rejected; bet > bankroll rejected; bet within limits accepted.
Baccarat natural	If Punto is dealt 8 or 9, no further cards drawn for either side.

Unit Testing Plan (unittest)

- Unit tests will be written in tests/ using Python's built-in unittest module. The key methods to test as they are implemented are:
-
- CasinoTakeAlgorithm.get_valid_takes(): Parameterised test cases using subTest() with the rules example and edge cases (empty table, table with one card, all cards equal).
- DeckCasinoGame.calculate_round_points(): Provide known collected piles; assert exact point totals.
- Hand.get_value(): Multiple test cases using subTest() with 10+ card combinations; assert correct integer return.

- Deck.deal(): After dealing n cards, len(deck.cards) == 52-n; dealing from empty deck raises ValueError, asserted with assertRaises().
- BettingSystem.calculate_payout(): Blackjack natural (bet=100) returns 250; standard win returns 200; loss returns 0.
- FileManager.save/load(): Write a full Deck Casino game state dict to a temp file using tempfile.TemporaryDirectory(), reload it, assert loaded dict equals saved dict.

8. Libraries and Other Tools

Library/ Tool	Purpose	Justification
PyQt6	Main GUI framework (widgets, signals/slots, QPainter for card rendering)	Specified as allowed. Rich widget set suitable for card game UI.
itertools (stdlib)	Subset enumeration in CasinoTakeAlgorithm	Standard library; combinations() gives all card subsets cleanly.
json (stdlib)	Save/load game state and bankroll	No external dependency; human-readable format; built into Python.
random (stdlib)	Deck shuffling (random.shuffle)	Standard library; appropriate for card shuffling simulation.
unittest (stdlib)	Unit test runner	Built into Python; no extra dependency; TestCase, subTest(), and assertRaises() cover all planned test scenarios. |

9. Schedule

Date	Phase	Tasks	Est. Time
23 Feb – 1 Mar	Setup & Core Model	Create GitLab repo and project skeleton. Implement Card (with dual hand/table values), Deck, Hand, Player. Write unit tests for these classes.	8-10 h
2 Mar – 8 Mar	Casino Take Algorithm	Implement CasinoTakeAlgorithm (subset enumeration). Implement DeckCasinoGame core loop (text-based UI). Unit tests for algorithm with rules example.	10-12 h
9 Mar – 15 Mar *Checkpoint 3*	Deck Casino GUI(Checkpoint 3)	Implement MainWindow, LobbyView, DeckCasinoView with PyQt6. Connect model to view. Manual multi-player Deck Casino is playable.	8-10 h
16 Mar – 22 Mar	AI Opponent + Save/Load	Implement AIOpponent strategy. Implement FileManager with full game-state JSON. Save/load mid-round.	10-12 h
23 Mar – 29 Mar	Blackjack + Baccarat Logic	Implement BlackjackGame, AIDealer (basic strategy table), BaccaratGame (third-card rules), BettingSystem.	8-10 h

30 Mar – 5 Apr *Checkpoint 6*	Blackjack & Baccarat GUI(Checkpoint 6)	BlackjackView and BaccaratView. At least 10 unit tests passing with pytest.	8-10 h
6 Apr – 12 Apr	Polish & Integration	Card animations (QTimer), chip visuals (QPainter), AI vs AI testing, edge-case bug fixes, refactoring.	6-8 h
13 Apr – 19 Apr	Final Testing & Demo Prep	Full system test, remaining bug fixes, inline code documentation, demo preparation.	4-6 h

Checkpoint 3 (by 20 Mar): Deck Casino playable end-to-end (text or GUI); Card, Deck, Hand, Player, CasinoTakeAlgorithm unit-tested.

Checkpoint 6 (by 10 Apr): All three games (Deck Casino, Blackjack, Baccarat) playable with player vs AI; at least 10 unit tests passing.

Total estimated effort: 62-78 hours.

10. References and Links

- **Python documentation** — docs.python.org (random module, json module, unittest).
- **PyQt6 documentation** — doc.qt.io/qtforpython-6/ (QMainWindow, QWidget, QPainter, QStackedWidget, signals and slots).
- **Blackjack Basic Strategy** — Wizard of Odds (wizardofodds.com/games/blackjack/strategy/calculator/). Reference for AIDealer strategy table.
- **Baccarat Third-Card Rule** — Wizard of Odds (wizardofodds.com/games/baccarat/basics/). Used for rule validation.
- **3 Card Poker Rules & Pay Table** — Wizard of Odds (https://wizardofodds.com/games/three-card-poker/). Used for 3 Card Poker rules and pay tables.
- **Aalto Basics in Programming Y2 Course Material** — e.g. Programming principles, OOP design, UI components and testing chapters.